

cuGMEC Parameter Table

This document corresponds to `src/cuGMEC_param.h`. You must recompile after changing these parameters.

The Example column shows only one possible way to write each parameter; it does not imply a recommended value.

Notation:

Notation	Meaning
<Species>	Ion , Alpha , Beam
<Field>	Phi , A , dNe , dTe , dPi , dPa , dPb
<MHDField>	Phi , A , dNe , dTe
<PICField>	dPi , dPa , dPb

Data Types and File Paths

Parameter	Allowed Values	Description	Notes	Example
mhdReal	float / double	MHD calculation precision.	MHD precision should be greater than or equal to PIC precision.	<code>using mhdReal = double;</code>
picReal	float / double	PIC calculation precision.	PIC precision should be less than or equal to MHD precision.	<code>using picReal = float;</code>
inputDir	String path	Input file directory.	For a run from scratch, only the <code>MHDCollocated</code> file and the optional <code>MHDStaggered</code> file are needed. For a continued run, the <code>MHDContinue</code> and <code>PICContinue</code> files are also needed. They come from the previous run's <code>outputDir/final</code> .	<code>const std::string inputDir = "/path/to/input";</code>
outputDir	String path	Output file directory.		<code>const std::string outputDir = "/path/to/output";</code>
MHDCollocated	.bin file name	Collocated MHD equilibrium file.		<code>const std::string MHDCollocated = "MHDCollocated.bin";</code>
MHDStaggered	.bin file name	Staggered MHD equilibrium file.	Required only when <code>ifStaggered=trueType</code> .	<code>const std::string MHDStaggered = "MHDStaggered.bin";</code>
<Species>PhaseSpaceMapping	.bin file name	Phase-space orbit mapping file.	Required only when <code>ifOutputPhaseSpaceFrequency=trueType</code> .	<code>const std::string IonPhaseSpaceMapping = "IonPhaseSpaceMapping.bin";</code>

Normalization Parameters

Parameter	Allowed Values	Description	Notes	Example
B0	Positive real number	Magnetic-field normalization scale.		<code>const double B0 = 4.921751144619735;</code>
L0	Positive real number	Length normalization scale.		<code>const double L0 = 6.595629295925759;</code>
VA0	Positive real number	Velocity normalization scale.		<code>const double VA0 = 8.864164667700194e+06;</code>
RH00	Positive real number	Left endpoint of the radial interval.		<code>const double RH00 = 0.08;</code>
RH01	Positive real number	Right endpoint of the radial interval.		<code>const double RH01 = 0.90;</code>
PSITMAX	Positive real number	Outermost toroidal magnetic flux (Wb/rad).		<code>const double PSITMAX = 18.868213504765762;</code>

Grid and Parallel Scale

Parameter	Allowed Values	Description	Notes	Example
hostNums	Positive integer	Number of MPI processes.		<code>const int hostNums = 4;</code>
devNums	Positive integer	Number of GPUs used by each MPI process.		<code>const int devNums = 4;</code>
gridNx	Positive integer	Number of radial grid points.		<code>const int gridNx = 256;</code>
gridNy	Positive integer	Number of field-aligned grid points.	Must be divisible by <code>hostNums * devNums</code> .	<code>const int gridNy = 32;</code>
gridNz	Positive integer	Number of toroidal grid points.		<code>const int gridNz = 96;</code>
ppcNums	Positive integer	Average number of particles per species in each grid cell.		<code>const int ppcNums = 256;</code>

Toroidal Range and Initial Perturbation

Parameter	Allowed Values	Description	Notes	Example
tubes	Positive integer	Simulate $1/\text{tubes}$ of the toroidal domain.		<code>const int tubes = 6;</code>
leftN	Nonnegative integer	Lower bound of the retained toroidal mode number in the $1/\text{tubes}$ toroidal domain.	The actual toroidal mode range is <code>tubes*[leftN,rightN]</code> , with spacing <code>tubes</code> .	<code>const int leftN = 1;</code>
rightN	Nonnegative integer	Upper bound of the retained toroidal mode number in the $1/\text{tubes}$ toroidal domain.		<code>const int rightN = 6;</code>
perturbLeftN	Nonnegative integer	Lower bound of the initial perturbation toroidal mode number in the $1/\text{tubes}$ toroidal domain.	The actual toroidal mode range is <code>tubes*[perturbLeftN,perturbRightN]</code> , with spacing <code>tubes</code> .	<code>const int perturbLeftN = 1;</code>
perturbRightN	Nonnegative integer	Upper bound of the initial perturbation toroidal mode number in the $1/\text{tubes}$ toroidal domain.		<code>const int perturbRightN = 6;</code>
perturbRadialIndex	1 to <code>gridNx</code>	Radial peak grid point of the initial Gaussian perturbation.		<code>const int perturbRadialIndex = 85;</code>
perturbWidth	Positive real number	Radial width of the initial Gaussian perturbation.		<code>const mhdReal perturbWidth = 0.04;</code>
perturbAmplitude	Real number	Normalized <code>Phi</code> amplitude of the initial Gaussian perturbation.		<code>const mhdReal perturbAmplitude = 2.5e-9;</code>

Filtering

Parameter	Allowed Values	Description	Notes	Example
<code>ifFilterN<MHDField></code>	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the MHD field.		<code>using ifFilterN_Phi = trueType;</code>
<code>ifFilterN_dP</code>	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifFilterN_dP = trueType;</code>
<code>removeN<MHDField></code>	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the MHD field.		<code>constexpr std::array<int, 1> removeN_Phi = {7};</code>
<code>removeN_dP</code>	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<int, 0> removeN_dP = {};</code>
<code>selectNM<MHDField></code>	<code>{ {N, leftM, rightM}, ... }</code>	Apply poloidal filtering to the specified toroidal mode numbers of the MHD field.		<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_Phi = {{{0, 0, 0}}};</code>
<code>selectNM_dP</code>	<code>{ {N, leftM, rightM}, ... }</code>	Apply poloidal filtering to the specified toroidal mode numbers of the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_dP = {{{0, 0, 1}}};</code>

MHD Physics Switches

Parameter	Allowed Values	Description	Notes	Example
<code>ifStaggered</code>	<code>trueType</code> / <code>falseType</code>	Whether to include the staggered grid.	Requires the <code>MHDStaggered</code> file.	<code>using ifStaggered = falseType;</code>
<code>ifNonlinearMHD</code>	<code>trueType</code> / <code>falseType</code>	Whether to include MHD nonlinear terms.		<code>using ifNonlinearMHD = trueType;</code>
<code>ifEparallel</code>	<code>trueType</code> / <code>falseType</code>	Whether to include the parallel electric-field term.		<code>using ifEparallel = trueType;</code>
<code>ifFLRMHD</code>	<code>trueType</code> / <code>falseType</code>	Whether to include the FLR effect from background thermal ions in the Poisson equation.		<code>using ifFLRMHD = falseType;</code>
<code>ifMaxwellStress</code>	<code>trueType</code> / <code>falseType</code>	Whether to include Maxwell stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifMaxwellStress = trueType;</code>
<code>ifReynoldsStress</code>	<code>trueType</code> / <code>falseType</code>	Whether to include Reynolds stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifReynoldsStress = trueType;</code>
<code>MaxwellStressCoef</code>	Real number	Maxwell stress coefficient.	Effective only when <code>ifMaxwellStress=trueType</code> .	<code>const mhdReal MaxwellStressCoef = 1.0;</code>
<code>ReynoldsStressCoef</code>	Real number	Reynolds stress coefficient.	Effective only when <code>ifReynoldsStress=trueType</code> .	<code>const mhdReal ReynoldsStressCoef = 1.0;</code>

Dissipation and Smoothing

Parameter	Allowed Values	Description	Notes	Example
<code>ifNablaPerp2<Field></code>	<code>trueType</code> / <code>falseType</code>	Whether to apply second-order perpendicular dissipation to the field.		<code>using ifNablaPerp2Phi = trueType;</code>
<code>perp2<Field></code>	Positive real number	Second-order perpendicular dissipation coefficient.		<code>const mhdReal perp2Phi = 1.0e-7;</code>
<code>ifNablaPara4<Field></code>	<code>trueType</code> / <code>falseType</code>	Whether to apply fourth-order parallel dissipation to the field.		<code>using ifNablaPara4Phi = trueType;</code>
<code>para4<Field></code>	Positive real number	Fourth-order parallel dissipation coefficient.		<code>const mhdReal para4Phi = 1.0e-6;</code>
<code>ifConvolveAligned</code>	<code>trueType</code> / <code>falseType</code>	Whether to apply Gaussian smoothing to the PIC perturbed pressure in the field-aligned direction.		<code>using ifConvolveAligned = trueType;</code>
<code>convolveTimes</code>	Positive integer	Number of smoothing repetitions each time smoothing is applied.		<code>const int convolveTimes = 1;</code>
<code>convolveSigmaMax</code>	Positive real number	Maximum standard deviation of the Gaussian kernel.		<code>const mhdReal convolveSigmaMax = 1.5;</code>
<code>convolveSigmaMin</code>	Positive real number	Standard-deviation threshold of the Gaussian kernel.		<code>const mhdReal convolveSigmaMin = 0.05;</code>
<code>convolveTSwitch</code>	Positive real number	Smoothing start time, normalized by Alfven time.		<code>const mhdReal convolveTSwitch = 200.0;</code>
<code>convolveDtSwitch</code>	Positive real number	Smoothing transition time, normalized by Alfven time.		<code>const mhdReal convolveDtSwitch = 25.0;</code>

PIC Physics Switches

Parameter	Allowed Values	Description	Notes	Example
ifFLRPIC	trueType / falseType	Whether to enable the FLR effect in the PIC part.		using ifFLRPIC = trueType;
ifNonlinearPIC	trueType / falseType	Whether to enable PIC nonlinear terms.		using ifNonlinearPIC = trueType;
gyroNums	Positive integer	Number of gyro-average points.		const int gyroNums = 4;

PIC Species Parameters

Parameter	Allowed Values	Description	Notes	Example
if<Species>	trueType / falseType	Whether to enable the corresponding ion species.		using ifIon = trueType;
<Species>Type	Maxwell / Slowing0 / Slowing1 / Slowing2 / Slowing3	Velocity distribution type.		const disType IonType = Maxwell;
<Species>Space	spaceReal / spaceUniform	Spatial sampling method: marker positions follow the physical spatial distribution or a uniform distribution.		const spaceType IonSpace = spaceReal;
<Species>Velocity	velocityReal / velocityUniform	Velocity sampling method: marker velocities follow the physical velocity-space distribution or a uniform distribution.		const velocityType IonVelocity = velocityUniform;
<Species>Mass	Positive real number	Ion mass, normalized by the proton mass.		const picReal IonMass = 2.5;
<Species>Char	Positive real number	Ion charge, normalized by the electron charge.		const picReal IonChar = 1.0;
<Species>Beta	Positive real number	Ion beta on the magnetic axis ($P/(B_0^2/(2\mu_0))$).		const picReal IonBeta = 0.037793721898356;
<Species>Vmin	Positive real number	Lower bound of the ion velocity, normalized by VA_0 .		const picReal IonVmin = 0.0135;
<Species>Vmax	Positive real number	Upper bound of the ion velocity, normalized by VA_0 .		const picReal IonVmax = 0.54;
<Species>Vb	Positive real number	Cutoff velocity of the slowing-down distribution.		const picReal BeamVb = 0.1;
<Species>DeltaV	Positive real number	Cutoff width of the slowing-down distribution.		const picReal BeamDeltaV = 0.1;
<Species>Lambda0	Positive real number	Center of Λ in the anisotropic distribution.		const picReal BeamLambda0 = 0.4;
<Species>DeltaLambda2	Positive real number	Square of the Λ width in the anisotropic distribution.		const picReal BeamDeltaLambda2 = 1.0 / (4.5 * 4.5);

Distribution Type	Formula
Maxwell	$f \sim n \cdot T^{(-3/2)} \cdot \exp[-m \cdot v^2 / (2 \cdot T)]$
Slowing0	$f \sim n / (v^3 + v_c^3)$
Slowing1	$f \sim n \cdot [1 + \operatorname{erf}((V_b - v) / \Delta v)] / (v^3 + v_c^3)$
Slowing2	$f \sim n \cdot \exp[-(\Lambda - \Lambda_0)^2 / \Delta \Lambda^2] / (v^3 + v_c^3)$
Slowing3	$f \sim n \cdot [1 + \operatorname{erf}((V_b - v) / \Delta v)] \cdot \exp[-(\Lambda - \Lambda_0)^2 / \Delta \Lambda^2] / (v^3 + v_c^3)$

Diagnostic Switches

Parameter	Allowed Values	Description	Notes	Example
ifDiagAmplitude	trueType / falseType	Whether to diagnose mode amplitude.		using ifDiagAmplitude = trueType;
ifDiagFrequency	trueType / falseType	Whether to diagnose mode frequency.		using ifDiagFrequency = trueType;
ifDiagEparallel	trueType / falseType	Whether to diagnose the parallel electric field.		using ifDiagEparallel = trueType;
ifDiagDensity	trueType / falseType	Whether to diagnose ion perturbed density.		using ifDiagDensity = trueType;
ifDiagDiffusivity	trueType / falseType	Whether to diagnose ion diffusivity.		using ifDiagDiffusivity = trueType;
ifDiagZFDrive	trueType / falseType	Whether to diagnose the zonal-flow drive source.		using ifDiagZFDrive = falseType;
ifCheckNaN	trueType / falseType	Whether to check for NaN.	If NaN is diagnosed, the program stops immediately and writes output.	using ifCheckNaN = trueType;

MHD Field Output Switches

Parameter	Allowed Values	Description	Notes	Example
ifOutputPhi	trueType / falseType	Whether to output Phi at multiple time points.		using ifOutputPhi = trueType;
ifOutputA	trueType / falseType	Whether to output A at multiple time points.		using ifOutputA = falseType;
ifOutputdNe	trueType / falseType	Whether to output dNe at multiple time points.		using ifOutputdNe = falseType;
ifOutputdTe	trueType / falseType	Whether to output dTe at multiple time points.		using ifOutputdTe = falseType;
ifOutputdPi	trueType / falseType	Whether to output dPi at multiple time points.		using ifOutputdPi = falseType;
ifOutputdPa	trueType / falseType	Whether to output dPa at multiple time points.		using ifOutputdPa = falseType;
ifOutputdPb	trueType / falseType	Whether to output dPb at multiple time points.		using ifOutputdPb = falseType;

Phase-space Output Parameters

Parameter	Allowed Values	Description	Notes	Example
gridE	Positive integer	Number of phase-space grid points in the E direction.		const int gridE = 96;
gridPphi	Positive integer	Number of phase-space grid points in the Pphi direction.		const int gridPphi = 128;
gridLambda	Positive integer	Number of phase-space grid points in the Lambda direction.		const int gridLambda = 48;
ppcPhase	Positive integer	Average number of particles per species in each phase-space grid cell.		const int ppcPhase = 2048;
ifOutputPhaseSpaceJacobian	trueType / falseType	Whether to output the phase-space Jacobian.	Output during the initialization stage.	using ifOutputPhaseSpaceJacobian = trueType;
ifOutputPhaseSpaceFrequency	trueType / falseType	Whether to output phase-space orbit frequencies.	Requires the <code><Species>PhaseSpaceMapping</code> file.	using ifOutputPhaseSpaceFrequency = falseType;
ifOutputPhaseSpaceF0	trueType / falseType	Whether to output the phase-space equilibrium distribution function.	Output during the initialization stage.	using ifOutputPhaseSpaceF0 = falseType;
ifOutputPhaseSpaceDeltaF	trueType / falseType	Whether to output the phase-space perturbed distribution function.		using ifOutputPhaseSpaceDeltaF = falseType;
ifOutputPhaseSpacePower	trueType / falseType	Whether to output phase-space wave-particle interaction power.		using ifOutputPhaseSpacePower = falseType;
gridVpara	Positive integer	Number of pitch-space grid points in the vpara direction.		const int gridVpara = 128;
gridVperp	Positive integer	Number of pitch-space grid points in the vperp direction.		const int gridVperp = 64;
ppcPitch	Positive integer	Average number of particles per species in each pitch-space grid cell.		const int ppcPitch = 2048;
ifOutputPitchSpaceJacobian	trueType / falseType	Whether to output the pitch-space Jacobian.	Output during the initialization stage.	using ifOutputPitchSpaceJacobian = trueType;
ifOutputPitchSpaceF0	trueType / falseType	Whether to output the pitch-space equilibrium distribution function.	Output during the initialization stage.	using ifOutputPitchSpaceF0 = falseType;
ifOutputPitchSpaceDeltaF	trueType / falseType	Whether to output the pitch-space perturbed distribution function.		using ifOutputPitchSpaceDeltaF = falseType;
ifOutputPitchSpacePower	trueType / falseType	Whether to output pitch-space wave-particle interaction power.		using ifOutputPitchSpacePower = falseType;

Time Advancement

Parameter	Allowed Values	Description	Notes	Example
ifContinue	trueType / falseType	Whether to start from continue files.	The <code>inputDir</code> directory must contain the <code>MHDContinue</code> and <code>PICContinue</code> continue files. They come from the previous run's <code>outputDir/final</code> .	using ifContinue = falseType;
continueSteps	Nonnegative integer	Number of steps already completed before resuming.	Must match the suffix of the continue file names.	const int continueSteps = 0;
dt	Positive real number	MHD time step.		const double dt = 0.02;
totalSteps	Positive integer	Total number of MHD steps in this run.		const int totalSteps = 20000;
ratioDt	Positive integer	Ratio of the PIC time step to the MHD time step.	PIC uses <code>dt * ratioDt</code> for each push.	const int ratioDt = 1;
sortSteps	Positive integer	Particle sorting interval.		const int sortSteps = 25;
diagSteps	Positive integer	Diagnostic sampling interval.		const int diagSteps = 1;
outputSteps	Positive integer	Field and phase-space output interval.		const int outputSteps = 2500;